Project Proposal Form
Institute of Information Technology (IIT)
University of Dhaka

Student's Name:	**Adnan Bin Wahid**		
Student's Roll:	**BSSE-1442**	**Phone:**	**01567937800**

TokenWise: Sustainable Context Optimization for Coding Agents

Project Description

TokenWise is a VS Code extension tool that helps AI coding assistants work more efficiently by reducing unnecessary code context before sending it to Large Language Models (LLMs). It intelligently identifies and retains only the parts of the repository relevant to the current task, reducing token usage, API costs, and unnecessary context noise. By sending cleaner and more focused context to the model, TokenWise also helps improve response quality and reduce hallucinations. In addition, the system estimates the approximate carbon footprint and energy savings achieved through this optimization process.

Problem Statement & Objectives

Modern coding agents spend a significant portion of their token budget on repository reading operations rather than reasoning or editing tasks [1]. As repositories grow larger, excessive context causes reduced reasoning quality, unnecessary interaction rounds, increased API cost, and slower development workflows. Existing coding assistants also provide little visibility into how much computational energy and environmental cost are associated with each AI-assisted request [2].

Additionally, most current context compression approaches either break syntactic structure through token-level pruning or rely on generic summarization techniques that remove important implementation details required for debugging and software engineering tasks [1]. Consequently, developers lack an integrated system that can intelligently optimize repository context while also measuring the sustainability impact of AI-assisted coding workflows.

Proposed Solutions

TokenWise establishes a Task-Aware Skimming → Context Pruning → Carbon Tracking cycle to optimize developer workflows. The tool functions as a high-performance middleware that bridges the gap between massive codebases and token-constrained LLMs.

1. Goal-Driven Hint Generation: The tool initiates the process by formulating an explicit natural language "Goal Hint" based on the user's current prompt. This hint acts as a semantic guide to differentiate between essential logic and irrelevant boilerplate for the coding agent [1].

2. Line-Level Neural Skimming Instead of using generic token-level compression, TokenWise employs a lightweight Neural Skimmer. This model analyzes the repository locally and assigns a relevance score to each line of code based on its alignment with the generated Goal Hint, ensuring task-specific focus [1].

3. Adaptive Context Pruning The system performs selective pruning by removing lines that fall below a specific relevance threshold. This module ensures that critical structural elements—such as import statements, class headers, and function signatures—are preserved to maintain the syntactic and semantic integrity of the codebase [1].

4. Multi-Benchmark Feature Fusion To enable accurate carbon estimation, the tool merges data from industry-standard sources, including performance, energy and model quality leaderboards. This creates a comprehensive dataset for training non-intrusive estimation models without requiring hardware sensors [2].

5. Prompt-Level Carbon Estimation TokenWise implements the SEAL reference framework to provide fine-grained carbon estimates. It utilizes a multi-feature input vector—including input/output tokens, model size, GPU type, and latency—to calculate energy consumption without requiring intrusive hardware access [2].

6. Phase-Specific Dual Regressor Engine The system captures the distinct energy profiles of the Prefill (input processing) and Decode (generation) phases using a two-tier regression strategy. It employs specialized machine learning models for high-precision estimation across both known model ranges and massive frontier models, ensuring a low error rate across the inference lifecycle [2].

7. Sustainability Dashboard & Feedback Loop The VS Code extension provides a real-time dashboard that displays:

- Token Savings: The percentage of the "context wall" removed via skimming.
- Carbon ROI: The estimated reduction in CO_2 emissions achieved by sending pruned context instead of raw files.

Reference:

1. Wang, Y., et al. (2026). *SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents.* arXiv:2601.16746v3.
2. Pathania, P., et al. (2026). *SEALing the Gap: A Reference Framework for LLM Inference Carbon Estimation.* arXiv:2603.02949v1.

Languages or Tools to be used:
Supervisor’s Name: Mridha Md. Nafis Fuad **Signature of the supervisor:** **Date: 30th October, 2025**
Proposal Presentation Feedback:

Midterm 1 Presentation Feedback: